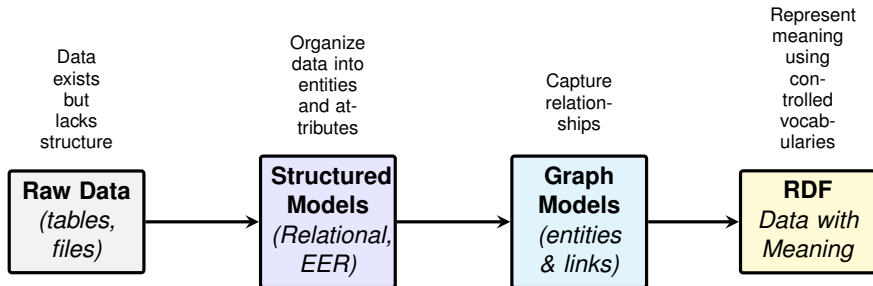


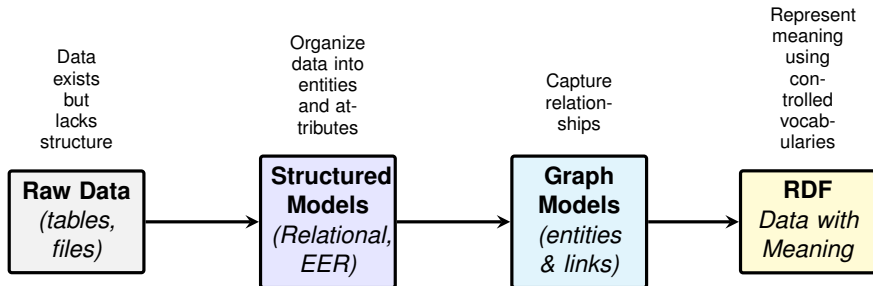
Data Graphs & RDF (Lecture 2)



Data Graphs & RDF – From Data to Meaning



Data Graphs & RDF – From Data to Meaning



Goal of the lecture

Understand how data models evolve from structural to semantic, and how RDF enables *interoperable, machine-understandable knowledge graphs*.

Agenda

- Data Models as Formal Systems
- Spectrum of Semantic Data Models
- Resource Description Framework (RDF)
- Reification in RDF
- RDF vs. Relational Model
- RDF as a Graph Data Model
- RDF Serializations

From formal abstractions to graph-based knowledge representation

Formal Systems (1/2)

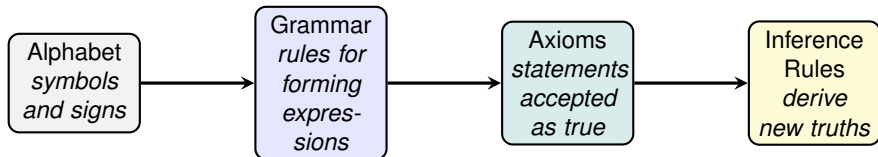
Key Idea

A formal system provides a **precise language** and a **logical mechanism** to represent and reason about knowledge in a consistent way.

Formal Systems (1/2)

Key Idea

A formal system provides a **precise language** and a **logical mechanism** to represent and reason about knowledge in a consistent way.



Formal Systems (2/2)

Definition (Formal System)

A formal system **F** is a 3-tuple $(\mathcal{V}, \mathcal{A}, \mathcal{I})$, where:

- $\mathcal{V}$ is the vocabulary of **F**; $\mathcal{V}$ is defined as a pair **(S,G)** where:
 - **S** is a finite set of symbols, known as the alphabet, which concatenate formulas, so that a formula is just a finite string of symbols taken from the alphabet.
 - **G** corresponds to the grammar of **F**; it consists of rules to form formulas from simpler formulas.
- $\mathcal{A}$ is a set of axioms, or axiom schemata, consisting of well-formed formulas.
- $\mathcal{I}$ is a set of inference rules. A **well-formed formula** that can be inferred from the axioms is known as a theorem of the formal system.

Data Models (1/2)

Key Idea

A **data model** provides an organized and formal representation of the world, defining *what* data can be stored, *how* it is structured, and *which rules* preserve its consistency.



Data Models (2/2)

Definition (Data Model)

A **data model** is a formal system composed of:

- **Data structure types** used to model real-world **entities**, and their **properties** and **relationships** among them.
- **Data manipulation** set of operations used to create and manage the data model structures.
- **Integrity constraints** set of rules to enforce consistency on the instances of the data model structures.

Evolution Data Models

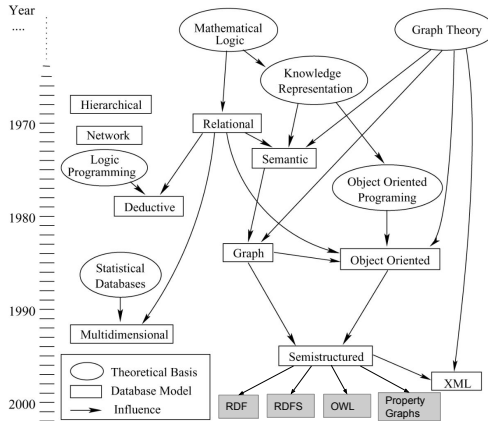
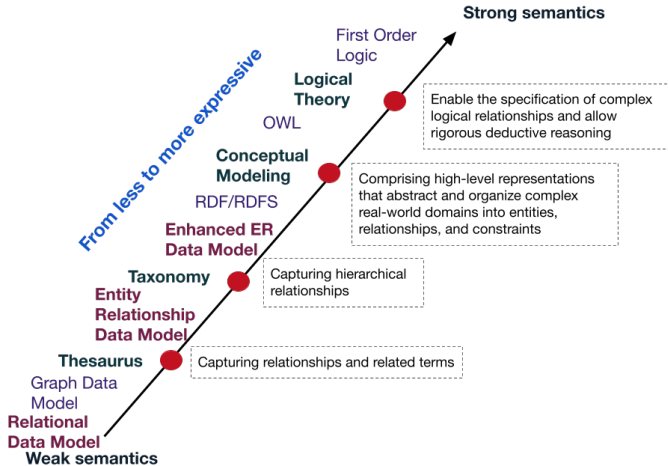


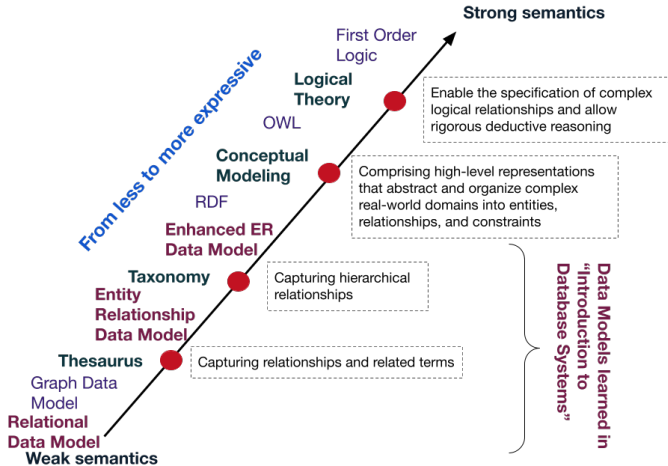
Fig. 1. Evolution of database models. Rectangles denote database models, arrows indicate influences, and circles denote theoretical developments. A time-line in years is shown on the left (based on a diagram by A. O. Mendelzon).

Figure: Based on figure about the evolution of data models [2].

Spectrum of Data Models (1/2)

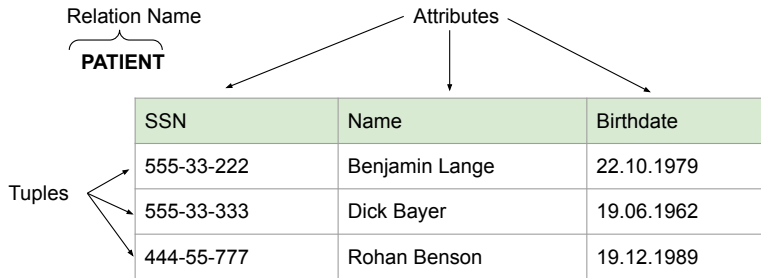


Spectrum of Data Models (2/2)



Relational Data Model (1/3)

The relational data model is based on the data structure **relation**
Informally, a relation is a table composed of columns and rows.



Only simple and mono-valued attributes can be represented.

Relational Data Model (2/3)- Relational Schema

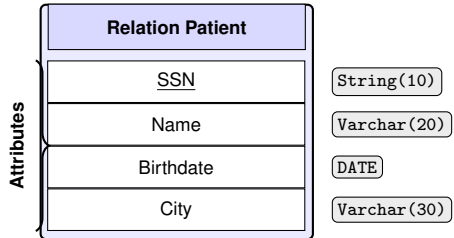
Idea

A **relation schema** specifies a table's structure: its name, its attributes (columns), and the **domains** (types) of the values.

Notation

For attributes $A_1, \dots, A_n$ with domains $D_1, \dots, D_n$:

$$R(A_1:D_1, A_2:D_2, \dots, A_n:D_n)$$



Patient(SSN:String(10), Name:Varchar(20),
Birthdate:DATE, City:Varchar(30))

Relational Data Model (3/3) — Integrity Constraints

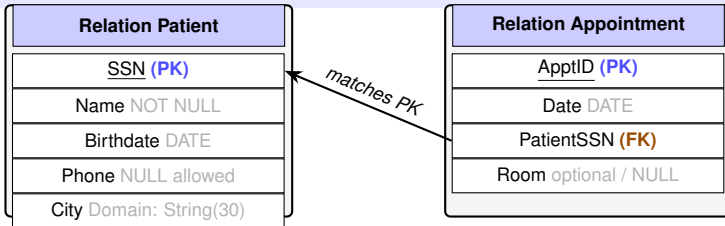
Purpose

Integrity constraints ensure that data stored in relations remains **accurate, consistent, and meaningful**.

Relational Data Model (3/3) — Integrity Constraints

Purpose

Integrity constraints ensure that data stored in relations remains **accurate, consistent, and meaningful**.



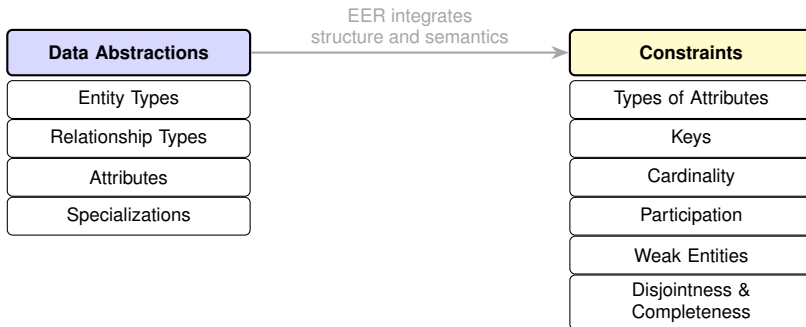
PK: Primary Key FK: Foreign Key

- **Optional attributes:** NULL allowed (e.g., Phone, Room); NOT NULL = mandatory.
- **Primary key:** unique, minimal, no NULL (e.g., Patient.SSN, Appointment.ApptID).
- **Referential integrity:** Appointment.PatientSSN must appear in Patient.SSN.
- **Domain constraints:** values must match their domains/types (e.g., DATE, String(30)).

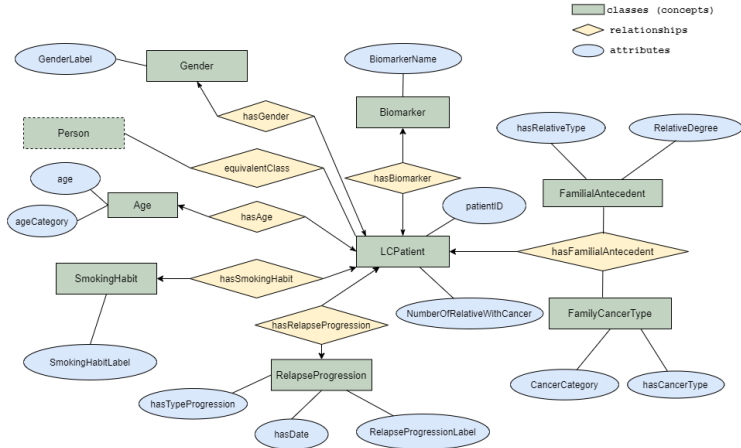
EER Data Model

Key Idea

The **Enhanced Entity–Relationship (EER) Model** extends the classical ER model to represent **domain knowledge** through richer **entity types**, **relationship types**, and **constraints**.



EER Schema- Example

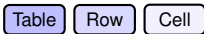


Types of Data Models: Classification by Typing

Data models can be classified by how strictly they enforce types and structure in representing data.

1. Strictly Typed (Structured)

- Each datum belongs to a predefined, fixed type.
- Schema changes require redesign.
- **Examples:** Relational, ER, Object-Relational.



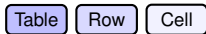
Structured representation: fixed schema of tables and attributes

Types of Data Models: Classification by Typing

Data models can be classified by how strictly they enforce types and structure in representing data.

1. Strictly Typed (Structured)

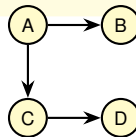
- Each datum belongs to a predefined, fixed type.
- Schema changes require redesign.
- **Examples:** Relational, ER, Object-Relational.



Structured representation: fixed schema of tables and attributes

2. Loosely Typed (Semi-Structured)

- Entities exist without strict categories or fixed schema.
- Relationships and attributes may evolve dynamically.
- **Examples:** Datalog, XML, RDF.



Flexible representation: graph data model

Example – Strongly Typed Data Models

Universe of Discourse

- **John Smith** is a Master student at **LUH**; he is 25 years old and studies **Informatics**.
- **Peter Haas** is a Master student at **UniBonn**; he studies **Economics**. Peter is **married** and has **one child**.

Example – Strongly Typed Data Models

Universe of Discourse

- **John Smith** is a Master student at **LUH**; he is 25 years old and studies **Informatics**.
- **Peter Haas** is a Master student at **UniBonn**; he studies **Economics**. Peter is **married** and has **one child**.

Conceptual View

- Real-world objects: *Students*, *Universities*, *Fields of Study*.
- Each object has properties with fixed data types:
 - Name: String, Age: Integer
 - University: String, Program: String

Example – Strongly Typed Data Models

Universe of Discourse

- **John Smith** is a Master student at **LUH**; he is 25 years old and studies **Informatics**.
- **Peter Haas** is a Master student at **UniBonn**; he studies **Economics**. Peter is **married** and has **one child**.

Conceptual View

- Real-world objects: *Students, Universities, Fields of Study*.
- Each object has properties with fixed data types:
 - Name: String, Age: Integer
 - University: String, Program: String

Representation in a Relation

Name	Age	University	Program
John Smith	25	LUH	Informatics
Peter Haas	NULL	UniBonn	Economics

Fixed schema: each attribute must conform to a predefined data type.

Example – Loosely Typed Data Models

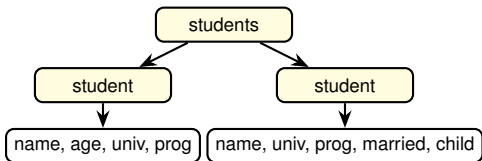
Semi-structured models such as **XML** or **RDF** allow entities to have *variable structures* and labels — data may omit or extend attributes without requiring a schema redesign [1].

Example – Loosely Typed Data Models

Semi-structured models such as **XML** or **RDF** allow entities to have *variable structures* and labels — data may omit or extend attributes without requiring a schema redesign [1]. **Representation in XML (Example from Slide 17)**

```
<students>
  <student>
    <name>John Smith</name>
    <age>25</age>
    <university>LUH</university>
    <program>Computer Science</program>
  </student>

  <student>
    <name>Peter Haas</name>
    <university>UniBonn</university>
    <program>Economics</program>
    <married>true</married>
    <children>1</children>
  </student>
</students>
```



Entities differ in structure and attributes —
no strict schema enforced.

Graphs as a Data Model

- Directed edge-labeled Graphs
- Heterogeneous Graphs
- Property Graphs

Graph Data Models

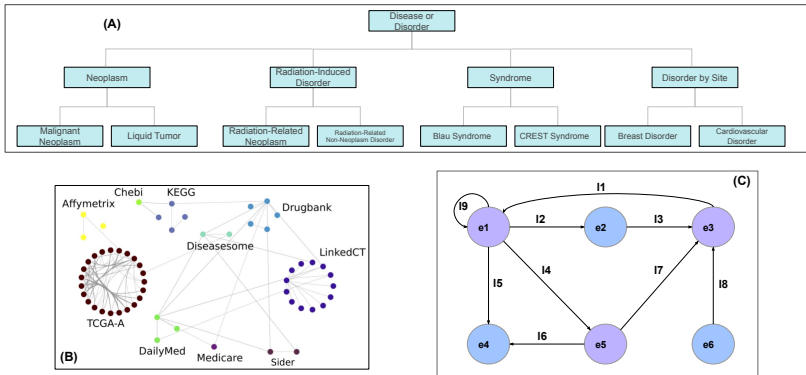
Definition (Graph Data Model [2])

A **graph data model** is a model where

- **Data structures** correspond to graphs,
- **Data manipulation** is expressed by graph-oriented operations and constructors, and
- **Integrity constraints** can be defined over the graph structure.

Graphs (from Lecture 1)

Intuitively, a graph is a set of vertices connected by edges



- (A) Tree or undirected graph in which any two vertices are connected by exactly one path.
- (B) Undirected graphs have edges that do not have a direction.
- (C) A directed labeled graph.

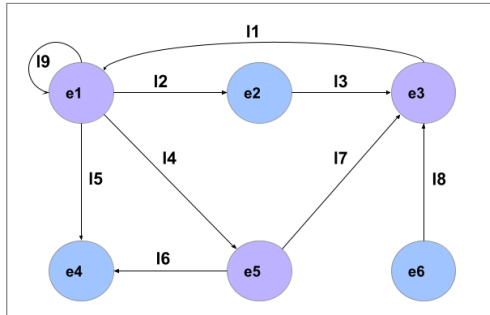
Directed Edge-Labeled Graphs

Definition (Directed Edge-Labeled Graph [4])

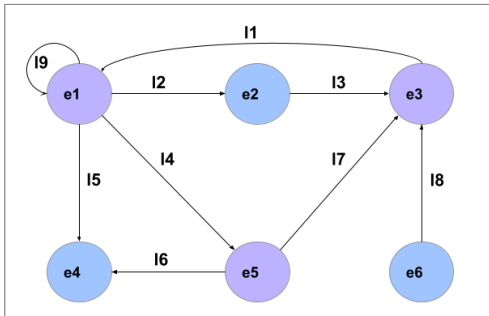
Given a countably set of infinite constants denoted by **Con**, a directed edge-labeled graph is a tuple $\mathbf{G}=(\mathbf{V},\mathbf{E},\mathbf{L})$, where:

- $\mathbf{V} \subseteq \mathbf{Con}$ is a set of nodes.
- $\mathbf{L} \subseteq \mathbf{Con}$ is a set of labels.
- $\mathbf{E} \subseteq \mathbf{V} \times \mathbf{L} \times \mathbf{V}$ is a set of labelled edges.

Example- Directed Edge-Labelled Graph



Example- Directed Edge-Labelled Graph



$G=(V,E,L)$:

- $V=\{e1,e2,e3,e4,e5,e6\}$
- $E=\{(e1,l9,e1),(e1,l2,e2),(e1,l4,e5),(e1,l5,e4),(e2,l3,e3), (e3,l1,e1),(e5,l7,e3),(e5,l6,e4),(e6,l8,e3)\}$
- $L=\{l1,l2,l3,l3,l5,l6,l7,l8,l9\}$
- $Con \supseteq V \cup L$

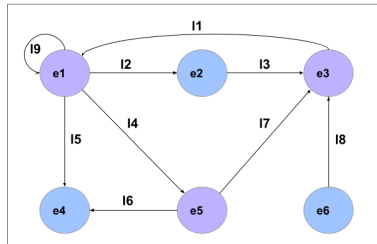
Heterogeneous Graphs

Definition (Heterogeneous Graph [4])

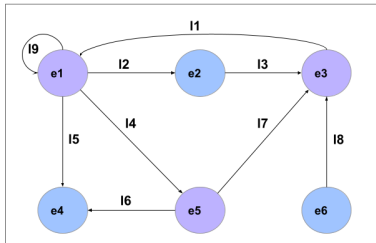
Given a countable set of infinite constants denoted by **Con**, a heterogeneous graph is a directed edge-labeled graph where each nodes are assigned to types. A heterogeneous graph is a tuple $G=(V,E,L,I)$, where:

- $V \subseteq \mathbf{Con}$ is a set of nodes.
- $L \subseteq \mathbf{Con}$ is a set of labels.
- $E \subseteq V \times L \times V$ is a set of labelled edges.
- $I: V \cup E \rightarrow 2^L$ maps each node id to a set of labels.

Example-Heterogeneous Graph



Example-Heterogeneous Graph



$G=(V,E,L):$

- $V=\{e1,e2,e3,e4,e5,e6\}$
- $E=\{(e1,l9,e1),(e1,l2,e2),(e1,l4,e5),(e1,l5,e4),(e2,l3,e3), (e3,l1,e1),(e5,l7,e3),(e5,l6,e4),(e6,l8,e3)\}$
- $L=\{l1,l2,l3,l3,l5,l6,l7,l8,l9\}$
- $Con \supseteq V \cup L$
- $I=\{(e1,\{t1\}),(e2,\{t2\}),(e3,\{t1\}),(e4,\{t2\}),(e5,\{t1\}),(e6,\{t2\})\}$
- $t1$ and $t2$ correspond to the types of the nodes.

Property Graphs

Definition (Property Graph [4])

Given a countably set of infinite constants denoted by **Con**, a property graph is a tuple $\mathbf{G}=(\mathbf{V},\mathbf{E},\mathbf{L},\mathbf{P},\mathbf{U},\mathbf{e},\mathbf{l},\mathbf{p})$, where:

- $\mathbf{V} \subseteq \mathbf{Con}$ is a set of nodes.
- $\mathbf{L} \subseteq \mathbf{Con}$ is a set of labels.
- $\mathbf{E} \subseteq \mathbf{V} \times \mathbf{L} \times \mathbf{V}$ is a set of labelled edges.
- $\mathbf{P} \subseteq \mathbf{Con}$ is a set of properties.
- $\mathbf{U} \subseteq \mathbf{Con}$ is a set of values.
- $\mathbf{e}: \mathbf{E} \rightarrow \mathbf{V} \times \mathbf{V}$ maps an edge id to a pair of nodes ids .
- $\mathbf{l}: \mathbf{V} \cup \mathbf{E} \rightarrow 2^{\mathbf{L}}$ is a set of node or edge ids .
- $\mathbf{p}: \mathbf{V} \cup \mathbf{E} \rightarrow 2^{\mathbf{P} \times \mathbf{U}}$ maps each node or edge id to a set of property-value pairs.

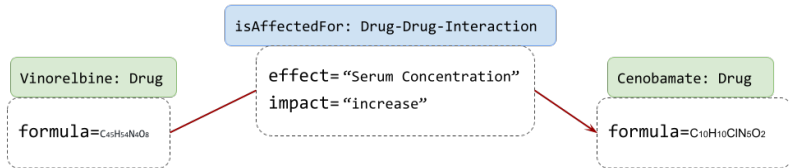
Example- Property Graph (1/2)

Let $\mathbf{G}=(\mathbf{V},\mathbf{E},\mathbf{L},\mathbf{P},\mathbf{U},\mathbf{e},\mathbf{I},\mathbf{p})$ be a property graph, where:

- $\mathbf{V} \supseteq \{\text{Vinorelbine, Cenobamate}\}.$
- $\mathbf{L} \supseteq \{\text{Drug, Drug-Drug-Interaction}\}.$
- $\mathbf{E} \supseteq \{\text{isAffectedFor}\}.$
- $\mathbf{P} \supseteq \{\text{effect, impact, formula}\}.$
- $\mathbf{U} \supseteq \{\text{"serum concentration", "increase"}\}.$
- $\mathbf{e}$ includes the mapping $\mathbf{e}(\text{isAffectedFor})=(\text{Vinorelbine, Cenobamate})$
- $\mathbf{I}$: includes the mappings: $\mathbf{I}(\text{Vinorelbine})=\{\text{Drug}\},$
 $\mathbf{I}(\text{Cenobamate})=\{\text{Drug}\}, \mathbf{I}(\text{isAffectedFor})=\{\text{Drug-Drug-Interaction}\}$
- $\mathbf{p}$ includes the mappings: $\mathbf{p}(\text{isAffectedFor})=\{(\text{effect, "serum concentration"}), (\text{impact, "increase"})\},$ $\mathbf{p}(\text{Vinorelbine})=\{(\text{formula, C}_{45}\text{H}_{54}\text{N}_4\text{O}_8)\}$ $\mathbf{p}(\text{Cenobamate})=\{(\text{formula, C}_{10}\text{H}_{10}\text{ClN}_5\text{O}_2)\}$

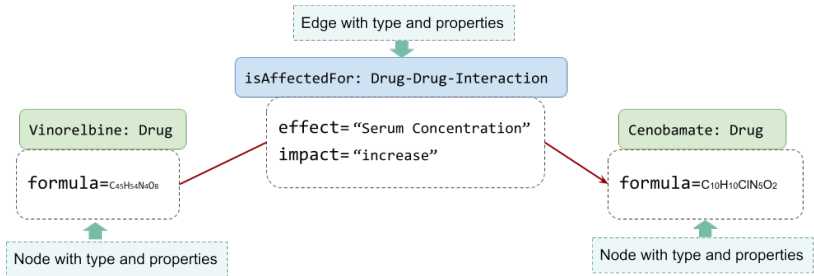
Example- Property Graph (2/2)

Graphical representation of property graph $G=(V,E,L,P,U,e,l,p)$ from slide 27



Example- Property Graph (2/2)

Graphical representation of property graph $G=(V,E,L,P,U,e,l,p)$ from slide 27



Resource Description Framework (RDF)

- **RDF as a Graph Data Model**

Connects data as **subject–predicate–object** triples.

- **W3C Standard**

Ensures interoperability and semantic consistency across datasets.

- **Main Features**

Global identifiers (URIs), schema flexibility, and machine interpretability.

- **Reification**

Allows statements *about statements* — enabling metadata and provenance.

- **RDF Serializations**

Turtle, RDF/XML, JSON-LD — same graph, different syntax.

RDF links data, enabling machines to understand relationships.

Resource Description Framework (RDF) (1/2)

RDF Overview:

- **RDF** is a flexible **data model** used for representing information about resources on the web.

Key Concepts:

- **Resource:**
 - A resource can be anything (e.g., **web pages**, **people**, **objects**, etc.).
 - Each resource is **uniquely identified** using a **URI** (Uniform Resource Identifier).
- **Description:**
 - Resources are described through their **properties** and **relationships** with other resources.
 - These descriptions form a **graph-like structure** of interconnected data.
- **Framework:**
 - RDF is a framework built on **web-based protocols** such as URIs, HTTP, XML, Turtle, and JSON.
 - It enables **seamless data interchange** across the web.

Resource Description Framework (RDF) (2/2)

The factual statement "*Vinorelbine treats lung cancer*" is represented in RDF as follows:

Vinorelbine	treats	Lung Cancer
Subject	Property	Object

Resource Description Framework (RDF) (2/2)

The factual statement "*Vinorelbine treats lung cancer*" is represented in RDF as follows:

Vinorelbine *treats* Lung Cancer
Subject Property Object

An RDF graph is a **directed edge-labelled graph**



RDF as a W3C Standard



RDF 1.1 Concepts and Abstract Syntax

W3C Recommendation 25 February 2014

This version:

<http://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>

Latest published version:

<http://www.w3.org/TR/rdf11-concepts/>

Previous version:

<http://www.w3.org/TR/2014/PR-rdf11-concepts-20140109/>

Previous Recommendation:

<http://www.w3.org/TR/rdf10-concepts/>

Editors:

[Richard Cyganiak](#), [DERI](#), [NUI Galway](#)

[David Wood](#), [3 Round Stones](#)

[Markus Lanthaler](#), [Graz University of Technology](#)

Previous Editors:

[Graham Klyne](#)

[Jeremy J. Carroll](#)

[Brian McBride](#)

RDF as a W3C Standard

What is a W3C Standard?

- A **W3C standard** is a technical specification created by the **World Wide Web Consortium (W3C)**.
- It ensures that web technologies are **interoperable** across platforms and devices.
- Examples include standards like **HTML**, **CSS**, **XML**, and **RDF**.

RDF as a W3C Standard

What is a W3C Standard?

- A **W3C standard** is a technical specification created by the **World Wide Web Consortium (W3C)**.
- It ensures that web technologies are **interoperable** across platforms and devices.
- Examples include standards like **HTML**, **CSS**, **XML**, and **RDF**.

Why is it important that RDF is a W3C Standard?

- **Interoperability:** RDF can be used across different systems for data sharing and integration.
- **Global Adoption:** RDF is widely supported and used by web tools, browsers, and organizations worldwide.
- **Stability:** W3C standards go through **rigorous testing**, ensuring RDF is robust and reliable for long-term use.
- **Compatibility:** RDF integrates seamlessly with other web standards like **XML**, **SPARQL**, and **OWL**.

W3C standards ensure a shared, interoperable Web of data.

Identifying Resources in RDF

Definition:

- A **Uniform Resource Identifier (URI)** is a compact sequence of characters that **uniquely identifies** a resource, whether abstract or physical.

Types of URIs:

- **Digital Libraries:**
 - **Books and publications** (e.g., **ISBN**, **ISSN**)
 - **Digital Object Identifier (DOI)** — persistent identifier for digital content.
- **Web:**
 - **URL** — identifier indicating the web protocol to access the resource (e.g., **https**, **ftp**).

URIs provide global, unambiguous identification — the backbone of the Semantic Web.

Uniform Resource Identifier (URI)

Examples of URIs:

Entity	URI
Vinorelbine	https://www.wikidata.org/entity/Q420532 http://dbpedia.org/resource/Vinorelbine http://www4.wiwiss.fu-berlin.de/sider/resource/drugs/5672 http://yago-knowledge.org/resource/Vinorelbine
Type	http://www.w3.org/1999/02/22-rdf-syntax-ns#type
Chemotherapy	https://www.wikidata.org/entity/Q974135
Treats	http://www.wikidata.org/prop/P2175
Breast Cancer	https://www.wikidata.org/entity/Q128581
Non-Small Lung Cancer	https://www.wikidata.org/entity/Q3658562

Representing Data in RDF

```
@prefix ex: <http://www.example.com/> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
@prefix owl: <http://www.w3.org/2002/07/owl#> .
```

Hospital_ID	City	#beds
Hospital1	Hannover	350
Hospital2	Bonn	300

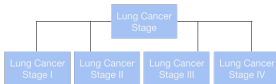
```
ex:Hospital    rdf:type          rdfs:Class .  
ex:Hospital1  rdf:type          ex:Hospital .  
ex:Hospital1  ex:located        ex:Hannover .  
ex:Hannover    rdf:type          ex:City .  
ex:Hospital1  ex:hasNumberBeds  "350"^^xsd:integer .  
ex:Hospital2  rdf:type          ex:Hospital .  
ex:Hospital2  ex:located        ex:Bonn .  
ex:Bonn        rdf:type          ex:City .  
ex:Hospital2  ex:hasNumberBeds  "300"^^xsd:integer .
```


Representing Data in RDF

```
@prefix ex: <http://www.example.com/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
```

Hospital_ID	City	#beds
Hospital1	Hannover	350
Hospital2	Bonn	300

```
ex:Hospital    rdf:type        rdfs:Class .
ex:Hospital1   rdf:type        ex:Hospital .
ex:Hospital1   ex:located      ex:Hannover .
ex:Hannover    rdf:type        ex:City .
ex:Hospital1   ex:hasNumberBeds "350"^^xsd:integer .
ex:Hospital2   rdf:type        ex:Hospital .
ex:Hospital2   ex:located      ex:Bonn .
ex:Bonn        rdf:type        ex:City .
ex:Hospital2   ex:hasNumberBeds "300"^^xsd:integer .
```



```
ex:LungCancerStage    rdf:type        rdfs:Class .
ex:LungCancerStageI   rdfs:subClassOf ex:LungCancerStage .
ex:LungCancerStageII  rdfs:subClassOf ex:LungCancerStage .
ex:LungCancerStageIII rdfs:subClassOf ex:LungCancerStage .
ex:LungCancerStageIV  rdfs:subClassOf ex:LungCancerStage .
```

RDF Graphs

Definition (RDF Graphs [3])

Given the set of infinite constants **Con** defined as the union of the infinitive sets U (set of URIs), B (set of Blank Nodes or existential variables), and L (set of literals).

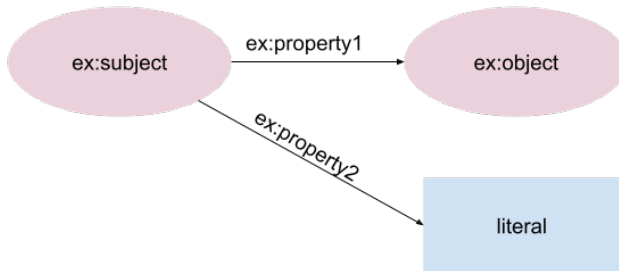
An RDF graph G is defined as a labelled directed graph $G=(V,E,L)$ where:

- $V \subseteq U \cup B \cup L$.
- E is a set of triples $t = (V_1, V_2, V_3)$:
 - V_1 is the subject of t and belongs to $U \cup B$.
 - V_2 is the property of t and belongs to U .
 - V_3 is the object of t and belongs to $U \cup B \cup L$.

An RDF graph G is ground iff G is **blank-node free**.

RDF as a Graph

```
@prefix ex: <http://www.example.org/> .  
ex:subject    ex:property1 ex:object .  
ex:subject    ex:property2 Literal .
```



Examples RDF Triples

subject	Vinorelbine	type	chemotherapy-drug
property	Vinorelbine	treats	breast-cancer
object	Vinorelbine	treats	non-small-lung-cancer

RDF Triples: using resources from DBpedia and Wikidata

```
@prefix dbpedia: <http://dbpedia.org/resource/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix wde: <http://www.wikidata.org/entity/> .
@prefix wdp: <http://www.wikidata.org/prop/> .
```

```
dbpedia:Vinorelbine    rdf:type      wde:Q974135 .
dbpedia:Vinorelbine    wdp:P2175      wde:Q128581 .
dbpedia:Vinorelbine    wdp:P2175      wde:Q3658562 .
```

Examples RDF Triples

subject	Vinorelbine	type	chemotherapy-drug
property	Vinorelbine	treats	breast-cancer
object	Vinorelbine	treats	non-small-lung-cancer

RDF Triples: using resources from DBpedia and Wikidata

```

@prefix dbpedia: <http://dbpedia.org/resource/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix wde: <http://www.wikidata.org/entity/> .
@prefix wdp: <http://www.wikidata.org/prop/> .

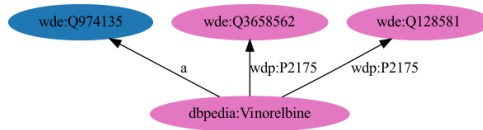
```

```

dbpedia:Vinorelbine    rdf:type          wde:Q974135 .
dbpedia:Vinorelbine    wdp:P2175         wde:Q128581 .
dbpedia:Vinorelbine    wdp:P2175         wde:Q3658562 .

```

RDF Graph: using resources from DBpedia and Wikidata



Blank Nodes in RDF (1/2)

Blank Nodes:

- Correspond to **existential variables**.
- Denote the existence of an **unknown resource** that makes the triple true.
- **Example:** If X denotes a blank node in the RDF triple $t = (X, p, o)$, this indicates that there exists at least one resource r such that (r, p, o) is true.

Blank nodes capture the existence of entities without giving them explicit identifiers.

Blank Nodes in RDF (2/2)

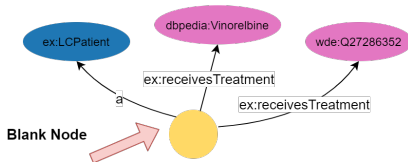
Blank Nodes versus NULL values in the Relational Model:

- A **blank node** is **not equivalent** to **NULL**.
- **NULL in Relational Model:**
 - A 'NULL' value for an attribute *ATT* in a relational triple (*key*, *ATT*, *value*) represents that it is **unknown whether the entity identified by *key* has a value** for the attribute *ATT*.
- **Blank Node in RDF:**
 - A blank node *X* for the property *ATT* in an RDF triple (*S*, *ATT*, *X*) states that *S* **has the property *ATT***, but the specific value is **unknown**.
RDF blank nodes express existence, while NULL expresses uncertainty.

Examples- Blank Nodes in RDF

A blank node denotes existence of an individual with specific attributes, but without an identification or reference.

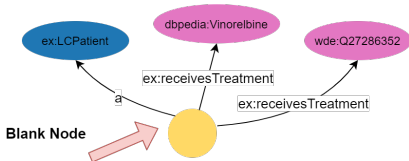
Example 1: There exists at least one lung cancer patient who takes Vinorelbine and Cenobamate



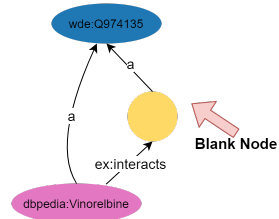
Examples- Blank Nodes in RDF

A blank node denotes existence of an individual with specific attributes, but without an identification or reference.

Example 1: There exists at least one lung cancer patient who takes Vinorelbine and Cenobamate



Example 2: Vinorelbine is a chemotherapy drug that interacts with at least one chemotherapy drug



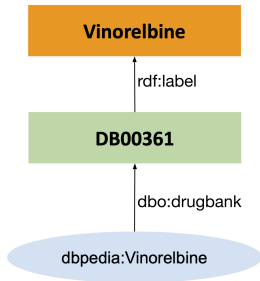
Examples- RDF Graphs (1/2)

Which of the following graphs are not RDF graphs?

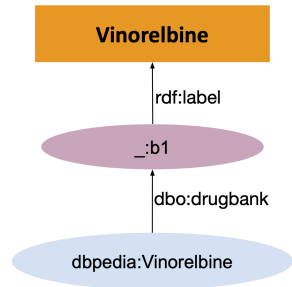
And Why?

Suppose that `_:b1` and `_:b2` are blank nodes, `Vinorelbine` and `DB00361` are literals, and `ex:DB00361` and `dbpedia:Vinorelbine` are URIs.

Graph 1:



Graph 2:



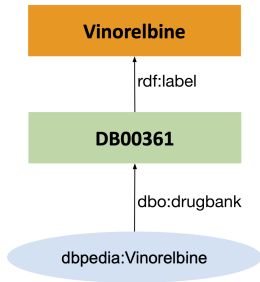
Examples- RDF Graphs (1/2)

Which of the following graphs are not RDF graphs?

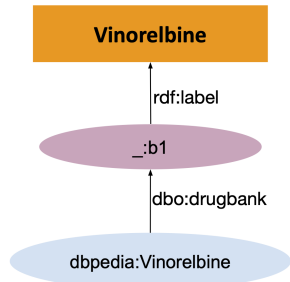
And Why?

Suppose that `_:b1` and `_:b2` are blank nodes, `Vinorelbine` and `DB00361` are literals, and `ex:DB00361` and `dbpedia:Vinorelbine` are URIs.

Graph 1:



Graph 2:



Graph 1 is not an RDF; literals cannot be in the subject position.

Graph 2 is a valid RDF; blank nodes can be in the subject and object positions.

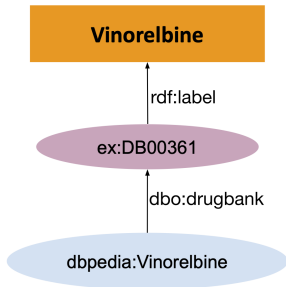
Examples- RDF Graphs (2/2)

Which of the following graphs are not RDF graphs?

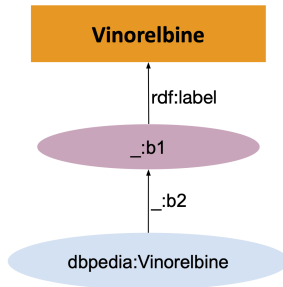
And Why?

Suppose that `_:b1` and `_:b2` are blank nodes, `Vinorelbine` and `DB00361` are literals, and `ex:DB00361` and `dbpedia:Vinorelbine` are URIs.

Graph 3:



Graph 4:



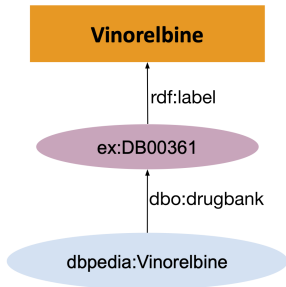
Examples- RDF Graphs (2/2)

Which of the following graphs are not RDF graphs?

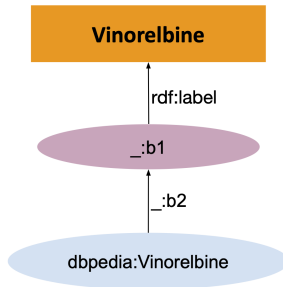
And Why?

Suppose that `_:b1` and `_:b2` are blank nodes, `Vinorelbine` and `DB00361` are literals, and `ex:DB00361` and `dbpedia:Vinorelbine` are URIs.

Graph 3:



Graph 4:



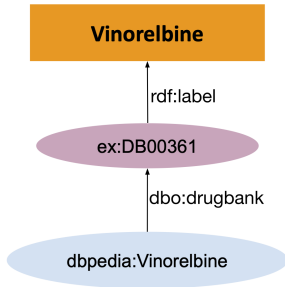
Examples- RDF Graphs (2/2)

Which of the following graphs are not RDF graphs?

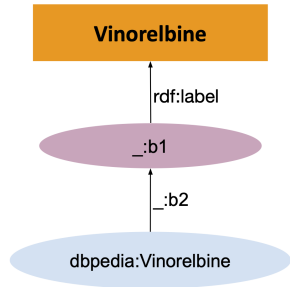
And Why?

Suppose that `_:b1` and `_:b2` are blank nodes, `Vinorelbine` and `DB00361` are literals, and `ex:DB00361` and `dbpedia:Vinorelbine` are URIs.

Graph 3:



Graph 4:



Graph 3 is a valid RDF; URIs can be in the subject and object positions.

Graph 4 is not an RDF; properties cannot be blank nodes.

Modeling with RDF

- John Smith is a Master student of LUH; he is 25 years old and studies Computer Science.
- Peter Haas is a Master student of UniBonn; he studies Economics. Peter is married and has one child.

Modeling with RDF

- John Smith is a Master student of LUH; he is 25 years old and studies Computer Science.
- Peter Haas is a Master student of UniBonn; he studies Economics. Peter is married and has one child.



- Peter Haas has age, but it is unknown; a **blank node** represents this situation.
- `a` represents **`rdf:type`**.

Relational Models versus RDF (1/2)

Id	Name	Major	University	Age	Married	Children
1	John Smith	Master Computer Science	LUH	25	NULL	NULL
2	Peter Haas	Master Economics	UniBonn	NULL	Y	1

Two meanings of **NULL** values in the Relational data model:

- Unknown value of a property (age) that all students should have.
- Unknown value of a property (children) that not all students have.

Relational Models versus RDF (2/2)

Blank nodes in RDF



How NULL values are represented in RDF:

- The value of age **exists** but it is unknown; it is represented with a **blank node**.
- An unknown value of a property (children) that not all the students have; it is represented with a **missing edge** in the graph.

N-ary Relationships in RDF (1/3)

Definition (N-ary Relationships)

Consider the classes $C_1, \dots, C_n$. A relation R between $C_1, \dots, C_n$ corresponds to the subset of the Cartesian Product of $C_1 \times \dots \times C_n$

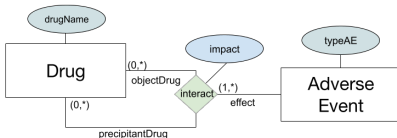
N-ary Relationships in RDF (1/3)

Definition (N-ary Relationships)

Consider the classes $C_1, \dots, C_n$. A relation R between $C_1, \dots, C_n$ corresponds to the subset of the Cartesian Product of $C_1 \times \dots \times C_n$

Example N-ary Relationship

ER Schema



Relational Schema

Precipitant Drug	Object Drug	Adverse Effect	Impact
Cenobamate	Vinorelbine	Serum Concentration	Decrease
Vinorelbine	Benperidol	Neurotoxic Activity	Increase

N-ary Relationships in RDF (2/3)

- RDF **natively** supports only **binary relationships** between two entities.
- To represent **N-ary relationships** (where $n > 2$), RDF introduces **helper nodes**.
- **Helper nodes** can be represented as either **blank nodes** or **fresh resources**.
- For each row in an **N-ary relation**, a **new helper node** is created.

RDF models N-ary relationships by introducing intermediate helper nodes linking multiple entities.

N-ary Relationships in RDF (3/3)

Definition (Representation of N-ary Relationships in RDF)

Let $R(C_1, C_2, \dots, C_n)$ be an N-ary relationship. In RDF, this is represented as a collection of binary relationships by introducing a **helper node** h . The relationships are expressed as:

$$(h, \text{hasParticipant}, c_1), (h, \text{hasParticipant}, c_2), \dots, (h, \text{hasParticipant}, c_n)$$

where h is a **blank node** or a **fresh resource** representing the N-ary relation instance, and each $(h, \text{hasParticipant}, c_i)$ represents the participation of class C_i in the relation.

Example: N-ary Relationships in RDF

Transformation of the N-ary relation into RDF

```
@prefix ex: <http://www.example.org/> .
@prefix rdf:<http://www.w3.org/1999/02/22-rdf-syntax-ns\#> .
@prefix wde: <http://www.wikidata.org/entity/> .
@prefix dbpedia: <http://dbpedia.org/resource/> .
```

N-ary Relation

Precipitant Drug	Object Drug	Adverse Effect	Impact
Cenobamate	Vinorelbine	Serum Concentration	Decrease
Vinorelbine	Benperidol	Neurotoxic Activity	Increase

```
ex:ddi1 rdf:type          ex:DDI.
ex:ddi1 ex:precipitantDrug wde:Q27286352.
ex:ddi1 ex:objectDrug     dbpedia:Vinoreline.
ex:ddi1 ex:adverseEffect  ex:SerumConcentration.
ex:ddi1 ex:impact         ex:Decrease.
```

```
ex:ddi2 rdf:type          ex:DDI.
ex:ddi2 ex:precipitantDrug dbpedia:Vinoreline.
ex:ddi2 ex:objectDrug     dbpedia:Benperidol.
ex:ddi2 ex:adverseEffect  ex:NeurotoxicActivity.
ex:ddi2 ex:impact         ex:Increase.
```

ex:ddi1 and ex:ddi2 are fresh resources or helper nodes

Relational Data Model vs RDF

Aspect	Relational Model	RDF Model
Structures	Relations: Subset of the Cartesian product of the domains of the relation	RDF triples: Subset of $UB \times U \times UBL$
Constraints	P1: Unique identification P2: Entity Integrity P3: Referential Integrity P4: Domain Constraint	R1: Domain Constraint R2: Existential values (Blank nodes)

Reification

Definition (Reification)

Reification refers to the expression of a concept within a language, enabling it to be treated and manipulated by the language.

Reification

Definition (Reification)

Reification refers to the expression of a concept within a language, enabling it to be treated and manipulated by the language.

Example-Reification:

Statement: "Anna says that the Earth is round."

- **Subject of the statement:** Anna
- **Predicate:** says
- **Discussed object:** "The Earth is round"

Reification in RDF

Definition (Reification in RDF)

Reification in RDF corresponds to factual statements about other factual statements.

Reification in RDF

Definition (Reification in RDF)

Reification in RDF corresponds to factual statements about other factual statements.

Representing Reification in RDF:

- RDF uses the class `rdf:Statement` to represent a statement.
- `rdf:subject` refers to the subject of the original statement.
- `rdf:predicate` refers to the property or predicate of the statement.
- `rdf:object` refers to the value or object of the statement.

Example1-Reification in RDF (1/3)



Shrek is a fictional ogre character; he thinks that “Ogres are like onions”

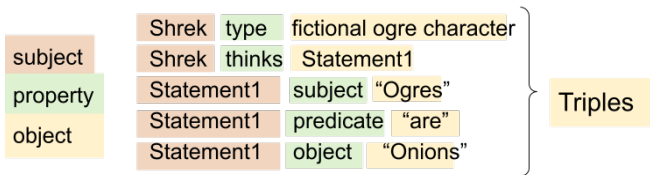
Factual statements

Shrek is a fictional ogre character

Shrek thinks statement1.

statement1 is “Ogres are like onions”

Factual statements in RDF

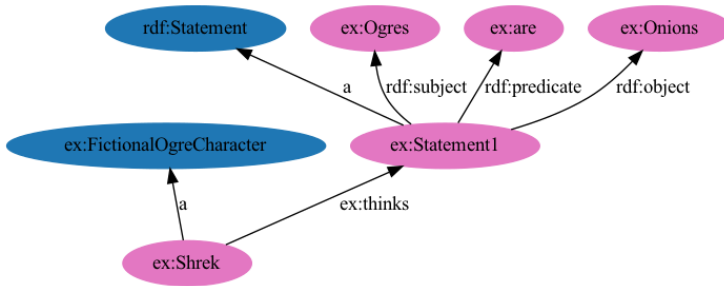


Example1-Reification in RDF (2/3)

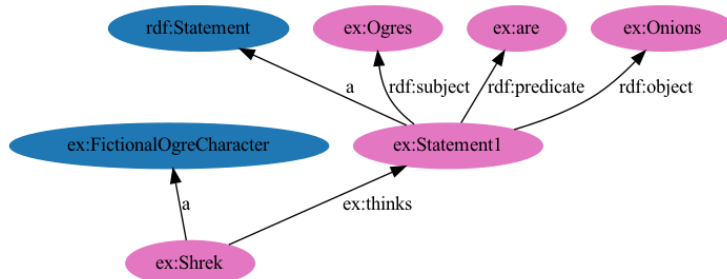


Shrek is a fictional ogre character; he thinks that
“Ogres are like onions”

Factual statements in an RDF graph



Example1-Reification in RDF (3/3)



Factual statements in an RDF graph

rdf:Statement: represents that an entity is a statement.

rdf:subject: represents that an entity is the subject of an RDF triple.

rdf:predicate: represents that an entity is the predicate of an RDF triple.

rdf:object: represents that an entity is the object of an RDF triple.

Example2-Reification in RDF (1/3)

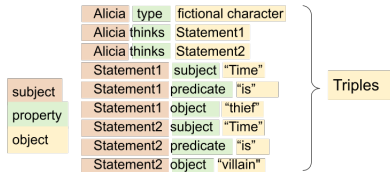


Alicia is a fictional character; she thinks that "Time is a thief and a villain".

Factual statements

Alicia is a fictional character.
Alicia thinks statement1 and statement2.
statement1 is "Time is a thief"
statement2 is "Time is a villain"

Factual statements in RDF

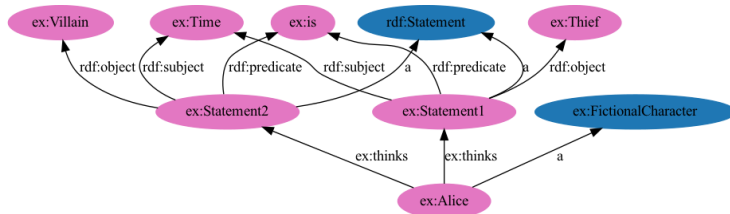


Example2-Reification in RDF (2/3)

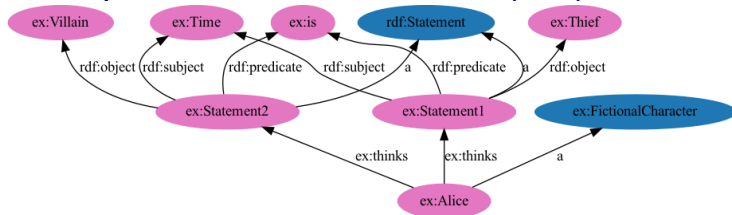


Alice is a fictional character; she thinks that "Time is a thief and a villain".

Factual statements in an RDF graph



Example2-Reification in RDF (3/3)



Factual statements in an RDF graph

`rdf:Statement`: represents that an entity is a statement.

`rdf:subject`: represents that an entity is the subject of an RDF triple.

`rdf:predicate`: represents that an entity is the predicate of an RDF triple.

`rdf:object`: represents that an entity is the object of an RDF triple.

Literals and Datatypes in RDF (1/2)

Literals in RDF:

- RDF literals can represent values such as strings, numbers, or dates.
- Typed literals are specified using XML Schema datatypes.

Namespace for Typed Literals:

`http://www.w3.org/2001/XMLSchema#`

Examples of RDF Literals with Datatypes:

```
"Leibniz University Hannover"^^<http://www.w3.org/2001/XMLSchema#string>  
"4500.60"^^<http://www.w3.org/2001/XMLSchema#float>  
"2022-10-21"^^<http://www.w3.org/2001/XMLSchema#date>
```

Literals and Datatypes in RDF (2/2)

Common RDF Datatypes:

- `xsd:string` – represents a string of characters.
- `xsd:integer`, `xsd:float` – represents numeric values.
- `xsd:date` – represents a calendar date.

RDF vs. Property Graphs - A Comparison (1/2)

- **RDF** (Resource Description Framework) is a W3C standard and a data model designed to represent knowledge graphs using triples.
- **Property Graphs** allow the direct representation of properties for both nodes and edges.
- **Key Concepts:**
 - RDF represents data in the form of subject-predicate-object triples.
 - Property Graphs use labeled, directed graphs with additional properties attached to nodes and edges.

RDF vs. Property Graphs - A Comparison (2/2)

	RDF	Property Graphs
Structure	Triples (Subject-Predicate-Object)	Labeled, Directed Graphs
Annotations	Blank Nodes, Reification	Properties on Nodes and Edges
Predicates	Predefined, Built-in Predicates	No Predefined Predicates
Data Model	Standardized by W3C	Flexible, No Standardization

RDF Serializations

Definition (RDF Serialization)

RDF Serialization refers to the process of encoding RDF data in a specific format so that it can be stored, transmitted, or exchanged between systems. RDF serializations ensure that RDF data is structured and represented consistently across platforms.

RDF Serializations

Definition (RDF Serialization)

RDF Serialization refers to the process of encoding RDF data in a specific format so that it can be stored, transmitted, or exchanged between systems. RDF serializations ensure that RDF data is structured and represented consistently across platforms.

Definition (Types of RDF Serialization)

Common RDF serialization formats include:

- **Turtle:** A human-readable, compact syntax for RDF.
- **N-Triples:** A simple, line-based format used for large datasets.
- **RDF/XML:** The original RDF syntax based on XML.
- **JSON-LD:** A JSON-based format for RDF, designed for easy integration with web data.

N-Triples (1/2)

Definition (N-Triples)

N-Triples is a line-based, plain text format for encoding RDF data. Each line in an N-Triples file represents a single RDF statement as a triple consisting of a subject, predicate, and object, ending with a period. This format is simple and designed for easy parsing of large datasets.

N-Triples (1/2)

Definition (N-Triples)

N-Triples is a line-based, plain text format for encoding RDF data. Each line in an N-Triples file represents a single RDF statement as a triple consisting of a subject, predicate, and object, ending with a period. This format is simple and designed for easy parsing of large datasets.

Syntax of N-Triples:

```
<N-Triple>      := <Subject> <Predicate> <Object> "."  
<N-Triples>     := <N-Triple> | <N-Triple> <N-Triples>  
<N-TripleFile> := <N-Triples>
```

N-Triples (1/2)

Definition (N-Triples)

N-Triples is a line-based, plain text format for encoding RDF data. Each line in an N-Triples file represents a single RDF statement as a triple consisting of a subject, predicate, and object, ending with a period. This format is simple and designed for easy parsing of large datasets.

Syntax of N-Triples:

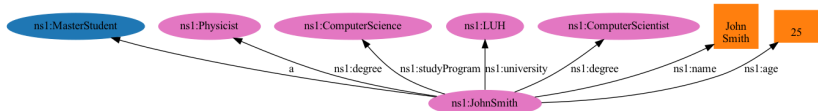
```
<N-Triple>      := <Subject> <Predicate> <Object> "."  
<N-Triples>     := <N-Triple> | <N-Triple> <N-Triples>  
<N-TripleFile>:= <N-Triples>
```

Example:

```
<http://example.com/JohnSmith> <http://example.com/age> "25"^^<http://www.w3.org/2001/XMLSchema#integer> .  
<http://example.com/JohnSmith> <http://example.com/name> "John Smith"^^<http://www.w3.org/2001/XMLSchema#string> .
```

N-Triples (2/2)

RDF Graph



N-Triples

```
<http://exam/JohnSmith> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://exam/MasterStudent> .
<http://exam/JohnSmith> <http://exam/university> <http://exam/LUH> .
<http://exam/JohnSmith> <http://exam/studyProgram> <http://exam/ComputerScience> .
<http://exam/JohnSmith> <http://exam/degree> <http://exam/ComputerScientist> .
<http://exam/JohnSmith> <http://exam/degree> <http://exam/Physicist> .
<http://exam/JohnSmith> <http://exam/age> "25"^^<http://www.w3.org/2001/XMLSchema#integer> .
<http://exam/JohnSmith> <http://exam/name> "John Smith"^^<http://www.w3.org/2001/XMLSchema#string> .
```

Turtle - Terse RDF Triple Language Notation

Definition (Turtle)

Representation of RDF triples using prefixes, blank nodes, and shorthand for common RDF structures.

Turtle Syntax

```
<TurtleFile>      := <Prefixes> <Statements>
<Prefix>          := @prefix <PrefixWord>: <Namespace> .
<Prefixes>        := <Prefix>| <Prefix> <Prefixes>
<Statements>      := <Statement> <Statements> | epsilon

<Statement>       := <Triples> .
<Triples>         := <Subject> <PredicateObjectList>
<PredicateObjectList> := <Predicate> <ObjectList> ; <PredicateObjectList> |
                           <Predicate> <ObjectList>
<ObjectList>      := <Object> , <ObjectList> | <Object>

<Subject>         := <IRI> | <BlankNode>
<Predicate>       := <IRI>
<Object>          := <IRI> | <Literal> | <BlankNode>
<BlankNode>       := _: <Identifier> | [ <PredicateObjectList> ]
<Literal>         := "literal"^^<Datatype> | "literal"
```

Example1- Turtle

```
@prefix ex: <http://example.com/>.
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
ex:JohnSmith a ex:MasterStudent ;
              ex:university ex:LUH ;
              ex:studyProgram ex:ComputerScience ;
              ex:degree ex:ComputerScientist ,
                        ex:Physicist ;
              ex:age "25"^^xsd:integer ;
              ex:name "John Smith"^^xsd:string .
```

Example2- Turtle

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:JohnSmith a ex:MasterStudent ;
  ex:studyProgram ex:ComputerScience ;
  ex:name "John Smith"^^xsd:string ;
  ex:university ex:LUH ;
  ex:degree ex:ComputerScientist ;
  ex:degree ex:Physicist ;
  ex:age "25"^^xsd:integer ;
  ex:hasTaken [ ex:hasCourseId ex:DB101 ;
                ex:hasSemester ex:SoSe2022 ] .
```

Transforming N-Triples into Turtle (1/2)

■ Key Transformation Steps:

■ Replacing Full URIs with Prefixed Names:

- Define prefixes for long URIs to enhance readability.

■ Grouping Triples with the Same Subject:

- Group all triples sharing the same subject to minimize repetition.

Transforming N-Triples into Turtle (1/2)

Key Steps in the Transformation Process:

■ Step 1: Define Prefixes:

- Identify frequently used namespaces and create prefixes for them.
- Example:

```
@prefix ex: <http://example.org/> .
```

■ Step 2: Group Triples by Subject:

- Combine multiple triples with the same subject into a single statement.
N-Triples

```
<http://example.org/subject> <http://example.org/predicate1> "object1" .  
<http://example.org/subject> <http://example.org/predicate2> "object2" .
```

Corresponding Turtle Representation:

```
ex:subject ex:predicate1 "object1" ;  
          ex:predicate2 "object2" .
```

Transforming N-Triples into Turtle (2/2)

Step 3: Convert Literals and Blank Nodes:

- **Literals:** In Turtle, literals can have language tags or data types.

N-Triples:

```
<http://example.org/subject> <http://example.org/predicate> "literal"^^<http://www.w3.org/2001/XMLSchema#string> .
```

Corresponding Turtle Representation:

```
ex:subject ex:predicate "literal"^^xsd:string .
```

- **Blank Nodes:**
- Represent blank nodes in Turtle more succinctly. N-Triples:

```
_:bnode <http://example.org/predicate> "literal" .
```

Corresponding Turtle Representation:

```
_:bnode ex:predicate "literal" .
```

Example1 - N-Triples and Turtle (1/2)

N-Triples

```
<http://example.com/JohnSmith> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example.com/MasterStudent> .
<http://example.com/JohnSmith> <http://example.com/university> <http://example.com/LUH> .
<http://example.com/JohnSmith> <http://example.com/studyProgram> <http://example.com/ComputerScience> .
<http://example.com/JohnSmith> <http://example.com/degree> <http://example.com/ComputerScientist> .
<http://example.com/JohnSmith> <http://example.com/degree> <http://example.com/Physicist> .
<http://example.com/JohnSmith> <http://example.com/age> "25"^^<http://www.w3.org/2001/XMLSchema#integer> .
<http://example.com/JohnSmith> <http://example.com/name> "John Smith"^^<http://www.w3.org/2001/XMLSchema#string> .
```

Corresponding Turtle Serialization

```
@base <http://example.com/> .
```

```
<JohnSmith> a <MasterStudent> ;
  <university> <LUH> ;
  <studyProgram> <ComputerScience> ;
  <degree> <ComputerScientist> ;
  <degree> <Physicist> ;
  <age> "25"^^xsd:integer ;
  <name> "John Smith"^^xsd:string .
```

Example2 - N-Triples and Turtle (1/2)

N-Triples

```
<http://example.com/JohnSmith> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example.com/MasterStudent> .
<http://example.com/JohnSmith> <http://example.com/university> <http://example.com/LUH> .
<http://example.com/JohnSmith> <http://example.com/studyProgram> <http://example.com/ComputerScience> .
<http://example.com/JohnSmith> <http://example.com/degree> <http://example.com/ComputerScientist> .
<http://example.com/JohnSmith> <http://example.com/degree> <http://example.com/Physicist> .
<http://example.com/JohnSmith> <http://example.com/age> "25"^^<http://www.w3.org/2001/XMLSchema#integer> .
<http://example.com/JohnSmith> <http://example.com/name> "John Smith"^^<http://www.w3.org/2001/XMLSchema#string> .
<http://example.com/JohnSmith> <http://example.com/hasTaken> :_b1.
_:b1 <http://example.com/hasCourseId> <http://example.com/DB101>.
_:b1 <http://example.com/hasSemester> <http://example.com/SoSe2022>.
```

Corresponding Turtle Serialization

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:JohnSmith a ex:MasterStudent ;
  ex:studyProgram ex:ComputerScience ;
  ex:name "John Smith"^^xsd:string ;
  ex:university ex:LUH ;
  ex:degree ex:ComputerScientist ;
  ex:degree ex:Physicist ;
  ex:age "25"^^xsd:integer ;
  ex:hasTaken [ ex:hasCourseId ex:DB101 ;
                ex:hasSemester ex:SoSe2022 ] .
```

Exercise

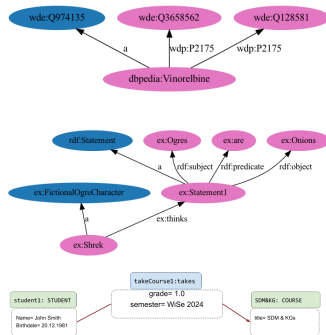
Participate **in the survey at Stud.IP.**

Summary

RDF is a data model and W3C standard to represent KGs

Reification in RDF graphs provides the possibility of representing statements about statements.

Property graphs enable to represent properties of nodes and edges.



References I



S. Abiteboul, P. Buneman, and D. Suci.

Data on the Web: From Relations to Semistructured Data and XML.

Morgan Kaufmann, 1999.



R. Angles and C. Gutiérrez.

Survey of graph database models.

ACM Comput. Surv., 40(1):1:1–1:39, 2008.



C. Gutiérrez, C. A. Hurtado, A. O. Mendelzon, and J. Pérez.

Foundations of semantic web databases.

J. Comput. Syst. Sci., 77(3):520–541, 2011.

References II



A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gao, R. Navigli, S. Neumaier, A. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. F. Sequeda, S. Staab, and A. Zimmermann. Knowledge graphs. *ACM Comput. Surv.*, 54(4):71:1–71:37, 2022.